

Национальный исследовательский университет ИТМО
Мегафакультет компьютерных технологий и управления
Факультет программной инженерии и компьютерной техники

Отчет по практическому заданию №3
по курсу «Языки программирования»

Вариант: 3

Выполнил студент группы Р4119

(подпись)

В.Ю. Суровикин

Руководитель

(подпись)

Ю. Д. Кореньков

17 декабря
2025 г.

Санкт-Петербург
2025

Оглавление

1. Цель работы	3
2. План работы	3
3. Ход работы	3
3.1. Описание виртуальной машины	3
3.2. Реализация транслятора	9
3.3. Примеры работы программы	14
4. Вывод	19
5. Исходные файлы	19

1. Цель работы

Реализовать формирование линейного кода в терминах некоторого набора инструкций посредством анализа графа потока управления для набора подпрограмм. Полученный линейный код вывести в мнемонической форме в выходной текстовый файл.

Вариант работы по 3 заданию представлен ниже:

Таблица 1. Исходные данные для 3 задания.

Вариант	Типизация	Формат инструкций VM	Банки памяти
23	Динамическая	Стековый	Code and data, stack

2. План работы

1. Составить описание виртуальной машины с набором инструкций и моделью памяти по варианту.
2. Реализовать модуль, формирующий образ программы в линейном коде для данного набора подпрограмм.
3. Доработать тестовую программу, разработанную в задании 2 для демонстрации работоспособности созданного модуля.

3. Ход работы

3.1. Описание виртуальной машины

Для того чтобы перейти к разработке транслятора, нам необходимо выполнить разработку описания виртуальной машины.

Первым делом необходимо определить регистры для проекта:

```

storage ip [32];      // instruction pointer (byte address in CODE space)
storage sp [32];      // stack pointer (byte address in STACK space, grows down)
storage bp [32];      // base/frame pointer for call frames (in STACK space)

storage status [32];  // 0 = OK, non-zero = runtime trap

// temporaries (64-bit because we operate on tagged values)
storage t0 [64];
storage t1 [64];
storage t2 [64];
    storage t3 [64];
    storage t4 [64];
    storage t5 [64];
    storage t6 [64];
    storage t7 [64];

// hidden IO port selector ("port"), used implicitly by IN/OUT
storage ioport [8];
// external device interface (environment-defined)
storage rin [8];
storage rout [8];

```

Рисунок 1 - Регистры виртуальной машины

Первые три регистра – служебные, пользователь может на них влиять лишь косвенно, через специальные команды:

1. ip – регистр, который хранит адрес текущей выполняемой команды.
2. sp – поскольку архитектура стековая, этот регистр хранит адрес вершины стека.
3. bp – регистр, который хранит указатель, использующийся для обращения к локальным переменным, а также передаваемым значениям в функцию.
4. status – регистр, отвечающий за корректное исполнение программы

Далее идут регистры, используемые для временного хранения вычислений в рамках одной инструкции, пользователь не имеет к ним доступа.

Последние три регистра нужны для системы ввода-вывода.

Далее определяются банки памяти, по заданию их 3:

```
memory:

    range code [0x0000 .. 0xffff] {
        cell = 8;
        endianness = little-endian;
        granularity = 1;
    }

    range dataMem [0x0000 .. 0xffff] {
        cell = 8;
        endianness = little-endian;
        granularity = 1;
    }

    range stackMem [0x0000 .. 0xffff] {
        cell = 8;
        endianness = little-endian;
        granularity = 1;
    }
```

Рисунок 2 - Банки памяти виртуальной машины

Также стоит отметить, что все регистры временного хранения - 64 битные, так как для обеспечения динамической типизации, они хранят в старшей части адреса специальный тэг, который определяет тип данных. Сами значения кодируются 32 битами.

Мнемоники приводится в отчете не будут, так как они копирует поведение мнемоник из реальных наборов инструкций. Подробно их просмотреть можно в исходных файлах.

В описанной архитектуре существуют поля инструкций от 8 до 64 бит для различных целей.

```
encode imm8 field = immediate [8];
encode imm16 field = immediate [16];
encode imm32 field = immediate [32];
encode imm64 field = immediate [64];
```

Рисунок 3 - Поля инструкций виртуальной машины

Были реализованы инструкции для условного и безусловного перехода.

```
// absolute unconditional jump
instruction jmp = { 0001 0000, imm16 as target } {
    ip = target;
};

// jump if top-of-stack is "false" (payload==0); pops 1 TV
instruction jz = { 0001 0001, imm16 as target } {
    t0 = stackMem:8[sp];
    sp = sp + 8;
    t1 = t0 & 0x00ffffffffffffff; // payload
    if t1 != 0 then
        ip = ip + 3; // non-zero => no jump
    else
        ip = target; // zero => jump
};

// jump if top-of-stack is "true" (payload!=0); pops 1 TV
instruction jnz = { 0001 0010, imm16 as target } {
    t0 = stackMem:8[sp];
    sp = sp + 8;
    t1 = t0 & 0x00ffffffffffffff; // payload
    if t1 != 0 then
        ip = target;
    else
        ip = ip + 3;
};
```

Рисунок 4 - Инструкции условного и безусловного перехода в виртуальной машине

Эти инструкции позволяют переходить к переданному адресу, есть возможность условного перехода, чтоб он совершался, если на вершине стека ноль или не ноль, в зависимости от инструкции. Значение с вершины стека удаляется.

Также была разработана инструкция для завершения программы.

```
instruction hlt = { 0000 0001 } {
    // stop execution
};
```

Рисунок 5 - Инструкция завершения программы в виртуальной машин

Реализованы также служебные команды, которые не вызываются пользователем, а генерируются при осуществлении трансляции.

```
// initialize SP (use before first PUSH). value is a STACK-space byte address.
instruction ldsp = { 0001 0110, imm16 as value } {
    sp = value;
    ip = ip + 3;
};

// initialize BP (optional). value is a STACK-space byte address.
instruction ldbp = { 0001 0111, imm16 as value } {
    bp = value;
    ip = ip + 3;
};
```

Рисунок 6 - Служебные инструкции виртуальной машины

Для стекового формата инструкций были разработаны специальные инструкции, непосредственно для управления стеком, стек растет вниз.

```
// push 64-bit tagged constant
instruction push = { 0001 1000, imm64 as value } {
    sp = sp - 8;
    stackMem:8[sp] = value;
    ip = ip + 9;
};

// duplicate top
instruction dup = { 0001 1001 } {
    t0 = stackMem:8[sp];
    sp = sp - 8;
    stackMem:8[sp] = t0;
    ip = ip + 1;
};

// drop top
instruction drop = { 0001 1010 } {
    sp = sp + 8;
    ip = ip + 1;
};

// swap top two
instruction swap = { 0001 1011 } {
    t0 = stackMem:8[sp];
    t1 = stackMem:8[sp + 8];
    stackMem:8[sp] = t1;
    stackMem:8[sp + 8] = t0;
    ip = ip + 1;
};

// load tagged value from DATA: pop addr, push data[addr]
instruction load = { 0010 0000 } {
    t0 = stackMem:8[sp];
    t0 = t0 & 0x00ffffff; // tagged address (treated as int)
    stackMem:8[sp] = dataMem:8[t0]; // payload
    ip = ip + 1;
};

// store tagged value into DATA: pop value; pop addr; data[addr]=value
instruction store = { 0010 0001 } {
    t0 = stackMem:8[sp]; // value
    t1 = stackMem:8[sp + 8]; // tagged address
    t1 = t1 & 0x00ffffff; // payload
    dataMem:8[t1] = t0;
    sp = sp + 16;
    ip = ip + 1;
};
```

Рисунок 7 - Инструкции для работы со стеком в виртуальной машине

Были реализованы инструкции для ввода и вывода.

```
// set hidden IO port register
instruction setio = { 0110 0000, imm8 as port } {
    ioport = port;
    ip = ip + 2;
};

// IN: read byte from current port (ioport), push TAG_INT payload=rin
// Environment is responsible for exposing the selected port value via RIN.
instruction inb = { 0110 0001 } {
    sp = sp - 8;
    stackMem:8[sp] = 0x0200000000000000 | (rin & 0xff);
    ip = ip + 1;
};

// OUT: pop value, write low byte to current port (ioport) via ROUT
instruction outb = { 0110 0010 } {
    t0 = stackMem:8[sp];
    sp = sp + 8;
    rout = t0 & 0xff;
    ip = ip + 1;
};
```

Рисунок 8 - Инструкции для работы с вводом/выводом в виртуальной машине

Также были созданы унарные и бинарные инструкции, инструкции сравнения, но здесь мы опустим их определение, так как оно интуитивно понятно. С их реализацией можно ознакомиться в исходном коде.

Главное - были реализованы инструкции вызова и возвращения из подпрограмм.

```
// call absolute; pushes (return_ip) and (old_bp) onto STACK
instruction call = { 0001 0011, imm16 as target } {
    sp = sp - 8;
    stackMem:8[sp] = ip + 3;    // return address
    sp = sp - 8;
    stackMem:8[sp] = bp;       // save caller bp
    bp = sp;                   // new frame base
    ip = target;
};

// return from call.
// Convention: callee places return value at [bp+16] before RET.
instruction ret = { 0001 0100 } {
    t0 = stackMem:8[bp];       // old bp
    ip = stackMem:8[bp + 8];    // return ip
    sp = bp + 16;              // drop frame, leave return value at top for caller
    bp = t0;
};
```

Рисунок 9 - Инструкции вызова и возврата из функции

Инструкции `call` и `ret` реализуют механизм вызова и возврата из подпрограмм с использованием стека и базового указателя кадра. При выполнении инструкции `call` виртуальная машина сохраняет в стеке адрес возврата (значение счетчика команд, указывающее на инструкцию, следующую за `call`) и базовый указатель вызывающей подпрограммы. После этого формируется новый стековый кадр: регистр `bp` устанавливается на вершину стека, а управление передается по абсолютному адресу вызываемой подпрограммы. Таким образом, каждый вызов подпрограммы создаёт собственный изолированный контекст выполнения.

Инструкция `ret` выполняет обратную операцию - разрушение стекового кадра и восстановление контекста вызывающей подпрограммы. Перед возвратом вызываемая подпрограмма помещает возвращаемое значение в стек по адресу $[bp + 16]$ в соответствии с принятым соглашением вызовов. При выполнении `ret` из стека извлекаются сохранённые значения базового указателя и адреса возврата, после чего указатель стека перемещается за пределы текущего кадра, оставляя возвращаемое значение доступным для вызывающей функции. В завершение восстанавливается прежний базовый указатель, и выполнение продолжается с сохраненного адреса возврата.

Таким образом целевая виртуальная машина была описана. Теперь перейдем к разработке непосредственно транслятора.

3.2. Реализация транслятора

Для того чтобы реализовать транслятор, нужно создать несколько ключевых структур. Сначала была создана структура, которая представляет очередную инструкцию.

```
struct Instruction {
    Mnemonic mnemonic{Mnemonic::nop};
    std::vector<Operand> operands; // printed as: <mnem> op1, op2, ...
    std::string comment;          // printed as: ; ...
};
```

Рисунок 10 - Структура представления инструкции

В ней хранится имя мнемоники и набор операндов в терминах целевой ВМ. Она не привязана к конкретному текстовому синтаксису, а является абстрактным представлением инструкции.

Далее была создана структура представления данных.

```
struct DataDef {
    DataDirective dir{DataDirective::db};
    // Values are stored as 64-bit and truncated/validated by emitter based on dir size.
    std::vector<std::uint64_t> values;
    std::string comment;
};
```

Рисунок 11 - Структура представления данных

Она может представлять собой либо конкретное литеральное значение, либо область памяти заданного размера.

Также была создана структура для хранения именованных фрагментов кода.

```
struct LineItem {
    std::optional<Label> label;

    // Exactly one payload per line (matches listing concept).
    std::variant<
        Instruction,
        DataDef,
        ReserveDef,
        CommentLine,
        std::shared_ptr<Times>
    > payload;
};
```

Рисунок 12 - Структура фрагмента линейного кода

Она хранит имя фрагмента, упорядоченный список инструкций и используется для создания пролога и эпилога программы, а также для каждого базового блока из потока управления.

Самая важная структура - образ программы целиком. Она является итоговым результатом модуля трансляции.

```
struct ProgramImage {  
    std::vector<Section> sections;  
  
    // Optional convenience metadata.  
    std::optional<std::string> entryLabel; // symbol name (before resolution)  
    std::optional<std::uint16_t> entryPoint; // address after resolution  
  
    // Filled by a later layout/link step (next parts of the task).  
    std::unordered_map<std::string, LabelInfo> labels;  
    std::vector<Fixup> fixups;  
};
```

Рисунок 13 - Структура образа программы

Перейдем к программному интерфейсу разработанного модуля. Программный интерфейс модуля генерации кода принимает на вход совокупность данных, сформированных на предыдущем этапе компиляции, а именно: набор подпрограмм программы; для каждой подпрограммы - граф потока управления (CFG); информацию о локальных переменных подпрограммы; сигнатуру подпрограммы (аргументы и тип возвращаемого значения).

```
// Main entry point:  
// - input: CFGs + signatures (CFGAnalysisResult from CFGBuild)  
// - output: ProgramImage (structures from task 1)  
CodeGenResult buildProgramImage(const CFGAnalysisResult& analysis);
```

Рисунок 14 - Интерфейс модуля трансляции

Данные структуры соответствуют определениям, разработанным в задании 2, и передаются в модуль генерации кода в виде абстрактных структур данных, не зависящих от конкретного синтаксиса исходного языка.

Модуль генерации образа программы преобразует информацию о подпрограммах, графах потока управления и локальных переменных в структуру данных, представляющую образ программы в памяти. Общий принцип работы модуля заключается в обходе графа потока управления каждой подпрограммы в порядке топологической сортировки, начиная с первого базового блока. Этот подход гарантирует корректное формирование линейного кода и правильную обработку всех переходов между блоками. Для каждой подпрограммы формируются следующие фрагменты кода: пролог, блоки для каждого базового блока и эпилог.

Пролог подпрограммы включает инструкции для сохранения контекста, формирования нового стекового кадра и выделения памяти под локальные переменные. Эти действия согласуются с соглашением вызовов, реализованным инструкциями `call` и `ret`. Каждый базовый блок генерирует фрагмент линейного кода, состоящий из инструкций, которые соответствуют операциям в дереве операций. Эпилог подпрограммы отвечает за размещение возвращаемого значения, восстановление базового указателя и возврат управления вызывающей подпрограмме.

Переходы между фрагментами формируются с использованием инструкций условных и безусловных переходов, операндами которых служат имена фрагментов, что упрощает кодогенерацию и повышает читаемость. Кроме того, модуль генерирует элементы данных, такие как области памяти для локальных переменных и литеральные значения, что позволяет точно разместить данные в памяти виртуальной машины.

Также важно отметить, что в модуле реализовано несколько встроенных функций для вывода значений в консоль, которые во время трансляции конвертируются напрямую в инструкции.

В тестовой программе модули используют созданные структуры данных для формирования образа программы и вывода ассемблерного листинга.

Для работы с программой был обновлен интерфейс, представленный в лабораторной работе 2. Программа имеет следующий формат запуска:

```
./ProgLangLab1 <input1> -o <out.asm> [--dump-cfg]
```

Рисунок 15 - Интерфейс тестовой программы

Теперь необходимо указать входной файл с кодом на целевом языке, а также опционально выбрать имя генерируемому файлу с ассемблерным листингом, или же генерируется в директории рядом с входным файлом файл с названием out.asm. Также можно указать дополнительный флаг, чтобы получить графическое отображение графов потока управления для указанного файла.

Пример ввода команды:

```
./ProgLangLab1 ../tests/trans_example.txt -o ../tests/trans_example.asm
```

Рисунок 16 - Пример использования

Для того, чтобы скомпилировать результат работы программы, в директории architecture находятся специальные скрипты для компиляции бинарного файла, его скачивания на хост и запуска на сервере с выводом результата в output файл.

3.3. Примеры работы программы

Для проверки работоспособности программы был написан комплексный пример, показывающий корректную трансляцию основных конструкций языка:

Листинг 1. Программа на целевом языке

```
function ab_func(x as int)
    dim b as int;
    b = 2;
    printInt(x + b);
    println();
end function

function isPos(x)
    if x > 0 then
        c = 'Y';
    else
        c = 'N';
    end if
    printChar(c);
    println();
end function

function main()
    v = 3;
    p = 3;
    r = 0;

    ab_func(v);

    while v > 1
        printInt(v);
        println();
        r = r + 1;
        v = v - 1;
    wend

    isPos(v);
```

```
    r = r + 5 - p;  
  
    printInt(r);  
end function
```

В этом примере рассматривается явное присваивание типов и неявное, вызовы функций, передача аргументов в них, создание локальных переменных, использование встроенных функций, ветвление, циклы и арифметические операции. Запустим тестовую программу и получим транслированный на ассемблер целевой виртуальной машины код.

Листинг 2. Листинг транслируемого ассемблерного кода

```
[section code, code]  
  
start:  
    ;  
    ldsp 0xFF00 ; init sp  
    ldbp 0xFF00 ; init bp  
    setio 0x0001 ; enable io  
    push 0x0200000000000000 ; return slot  
    call fn_main ; call main  
    drop ; drop main return  
    hlt  
fn_ab_func:  
    ;  
    push 0x0200000000000000 ; local init  
    push 0x0200000000000000 ; local init  
    jmp fn_ab_func_B0_entry ; enter body  
fn_ab_func_B0_entry:  
    ;  
    push 0x0200000000000002 ; int  
    stfp -8 ; store local b  
    ldfp 24 ; load param x  
    ldfp -8 ; load local b  
    add  
    push 0x0200000000000030 ; '0'  
    add ; to ascii digit
```

```

outb ; builtin printInt(0..9)
push 0x0200000000000000 ; printInt ret
drop ; drop expr result
push 0x040000000000000A ; newline
outb ; builtin println
push 0x0200000000000000 ; println ret
drop ; drop expr result
fn_ab_func_B1_exit:
    ;
push 0x0200000000000000 ; default return
stfp 16 ; write return slot
ret
fn_isPos:
    ;
push 0x0200000000000000 ; local init
push 0x0200000000000000 ; local init
jmp fn_isPos_B0_entry ; enter body
fn_isPos_B0_entry:
    ;
ldfp 24 ; load param x
push 0x0200000000000000 ; int
gt
jnz fn_isPos_B3_if_then ; if true
jmp fn_isPos_B4_if_else ; if false
fn_isPos_B4_if_else:
    ;
push 0x040000000000004E ; char
stfp -8 ; store local c
jmp fn_isPos_B2_if_join ; edge
fn_isPos_B3_if_then:
    ;
push 0x0400000000000059 ; char
stfp -8 ; store local c
fn_isPos_B2_if_join:
    ;
ldfp -8 ; load local c
outb ; builtin printChar
push 0x0200000000000000 ; printChar ret
drop ; drop expr result
push 0x040000000000000A ; newline
outb ; builtin println
push 0x0200000000000000 ; println ret

```



```

drop ; drop expr result
fn_isPos_B1_exit:
    ;
push 0x0200000000000000 ; default return
stfp 16 ; write return slot
ret
fn_main:
    ;
push 0x0200000000000000 ; local init
push 0x0200000000000000 ; local init
push 0x0200000000000000 ; local init
push 0x0200000000000000 ; local init
jmp fn_main_B0_entry ; enter body
fn_main_B0_entry:
    ;
push 0x0200000000000003 ; int
stfp -24 ; store local v
push 0x0200000000000003 ; int
stfp -8 ; store local p
push 0x0200000000000000 ; int
stfp -16 ; store local r
ldfp -24 ; load local v
push 0x0200000000000000 ; return slot
call fn_ab_func ; call fn_ab_func
stfp -32 ; spill call ret
drop ; drop arg
ldfp -32 ; reload call ret
drop ; drop expr result
fn_main_B2_while_cond:
    ;
ldfp -24 ; load local v
push 0x0200000000000001 ; int
gt
jnz fn_main_B3_while_body ; if true
jmp fn_main_B4_while_after ; if false
fn_main_B4_while_after:
    ;
ldfp -24 ; load local v
push 0x0200000000000000 ; return slot
call fn_isPos ; call fn_isPos
stfp -32 ; spill call ret
drop ; drop arg

```

```

ldfp -32 ; reload call ret
drop ; drop expr result
ldfp -16 ; load local r
push 0x0200000000000005 ; int
add
ldfp -8 ; load local p
sub
stfp -16 ; store local r
ldfp -16 ; load local r
push 0x0200000000000030 ; '0'
add ; to ascii digit
outb ; builtin printInt(0..9)
push 0x0200000000000000 ; printInt ret
drop ; drop expr result
fn_main_B1_exit:
    ;
push 0x0200000000000000 ; default return
stfp 16 ; write return slot
ret
fn_main_B3_while_body:
    ;
ldfp -24 ; load local v
push 0x0200000000000030 ; '0'
add ; to ascii digit
outb ; builtin printInt(0..9)
push 0x0200000000000000 ; printInt ret
drop ; drop expr result
push 0x040000000000000A ; newline
outb ; builtin println
push 0x0200000000000000 ; println ret
drop ; drop expr result
ldfp -16 ; load local r
push 0x0200000000000001 ; int
add
stfp -16 ; store local r
ldfp -24 ; load local v
push 0x0200000000000001 ; int
sub
stfp -24 ; store local v
jmp fn_main_B2_while_cond ; edge

```

[section data, dataMem]

Как можно увидеть из результата, все конструкции языка были корректно транслированный в ассемблерное представление. Для того чтобы проверить верность трансляции, скомпилируем и запустим на удаленном сервере получившийся листинг.

```
Task ExecuteBinaryWithInput started with id 9bf445a2-4e3e-4dd4-874f-0b18c633bcee
Waiting for completion...
Here is task output interpreted with UTF8:
5
3
2
Y
4
```

Рисунок 17 - Результат работы программы

Сервер выдал корректный ответ, который соответствует написанному на целевом языке коду.

4. Вывод

В ходе выполнения 3 лабораторной работы была разработана стековая виртуальная машина с динамической типизацией по варианту и реализован модуль трансляции, формирующий линейный код (в мнемонической форме) на основе графов потока управления, полученных на предыдущем этапе. Проведенное тестирование подтвердило корректность генерации кода для основных конструкций языка (арифметика, логика, ветвления, циклы, вызовы функций), а также работоспособность полученного результата при запуске.

5. Исходные файлы

Исходные файлы проекта: репозиторий [Электронный ресурс]. — GitHub. — Режим доступа: https://github.com/Vsevomies/ProgLang_lab1/tree/lab3 (дата обращения: 13.12.2025).